

04

Security & Privacy

Threat model, Firestore rules patterns, API hardening, private document handling, credential hygiene, and Data Privacy Act alignment. Each item is tied to an open finding and a test case.

Revision	Rev 2.0 — 1 October 2026
Supersedes	Security sections of the Firebase Setup Plan (Steps 5, 7, 10, 11) and the "unsigned preset is acceptable" note
Stack baseline	React + Firebase Spark (Auth, Firestore, Hosting) + Cloudinary Free + Vercel Hobby API + GitHub Actions. No paid plan, no credit card.
Audience	Whole team; required reading for anyone touching rules, the API, uploads or admin pages
Team	Andresa (Scrum Master) · Carillo · Laurente · Prades · Solpico

READ THIS FIRST

Three security findings stand between TrustHome PH and a defensible release. F-05: overlapping bookings can be confirmed in the deployed app. F-16: verification documents are reachable by anyone who has the URL. F-01: a service-account key was exposed and has not been confirmed revoked. All three now have a free fix: the trusted API, Cloudinary authenticated assets with expiring links, and per-consumer service accounts. This document explains each fix, gives reusable rules patterns, and lists what the Data Privacy Act of 2012 expects when a platform stores government IDs. The old guidance that "an unsigned public document URL is acceptable for an MVP" is withdrawn.

CONTENTS

- | | | | |
|-----------------------------|--|---|--|
| 1. Security principles | 4. Trusted API hardening | 6. Credentials (F-01) | 8. Data Privacy Act of 2012 (RA 10173) alignment |
| 2. Threat model | 5. Private verification documents (F-16) | 7. Web app hardening (Firebase Hosting) | 9. Security release gate |
| 3. Firestore rules patterns | | | |

Document set: 01 Product Requirements & Scope · 02 System Architecture & Data Design · 03 Infrastructure, Setup & Deployment · **04 Security & Privacy** · 05 UI/UX Design System · 06 Roadmap & Release Plan · 07 QA & Testing Strategy

1. Security principles

1. **Deny by default.** Every rule starts closed. The last rule in `firestore.rules` denies everything.
2. **The server decides anything that matters.** Money, time slots, trust and identity are decided by rules or the trusted API, never by UI state.
3. **Fail closed.** If a profile cannot be read, treat the user as unauthorized and offer a retry (FR-019).
4. **Least privilege.** Each person, service account and key gets the minimum access it needs, and nothing shared.
5. **Minimize data.** Do not collect what you do not need. Do not keep it longer than you need it. Do not put private data in documents anyone else can read.
6. **Test around the UI.** A security test is valid only if it bypasses the interface: direct SDK writes, raw HTTP, another account's token.

2. Threat model

#	Threat	Asset	Current control	Gap / finding	Fix (version)	Tests
T1	Owner confirms two overlapping bookings (race or modified client)	Booking integrity, renter money	Client transaction; rules deny direct <code>Confirmed</code>	F-05: callable not deployed (needs Blaze)	Trusted API transaction + lock (v1.4)	TC-API-005/006, TC-SEC-002
T2	Stranger opens an owner's ID or land title	Sensitive personal information	Not linked publicly; folders + tags	F-16: public Cloudinary URLs; legacy URLs in listing docs	Authenticated assets + 5-min links + migration (v1.4)	TC-DOC-001...009, TC-VER-006
T3	Leaked admin credential used to write anything	Whole database	Key untracked locally	F-01: revocation and Git history unverified	Per-consumer accounts, rotate, scan (v1.4 day 1)	TC-OPS-001/002
T4	User escalates own role to admin	Admin functions	Rules block role/status edits	Live test pending	Run TC-AUTH-010	TC-AUTH-010, TC-ADM-007
T5	Exact address scraped from public data	Physical safety	<code>listingPrivate</code> + 5-char geohash	Legacy listings not migrated	Run address migration after backup	TC-MAP-007/008/011/012
T6	Fake listings / fake verification	Marketplace trust	Admin plausibility review, two documents	Human error	Review checklist (\$6), reviewer name stored	TC-VER-*
T7	Self-booking to inflate trust	Ranking fairness	Completed-only + 3-in-30-days flag	Owner cannot confirm without the API (good)	Keep confirm server-side	TC-FAIR-003/006

#	Threat	Asset	Current control	Gap / finding	Fix (version)	Tests
T8	Notification spoofing / reading others' feeds	Privacy, phishing	Feed boundaries in rules	Live matrix not run	Run notification matrix	TC-NOTIF-004/006/007/009
T9	Quota exhaustion (abuse or bug) takes the app down	Availability	Spark hard limits	No monitoring	Quota checks (03 §10), referrer-restricted API key	TC-OPS-005
T10	Email harvesting from <code>users</code> collection	Personal data	—	Check: old draft rule allowed public read of <code>users</code>	Split public profile fields from private ones (§3.3)	TC-SEC-005 + new check

3. Firestore rules patterns

3.1 Helpers (top of `firestore.rules`)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    function signedIn() { return request.auth != null; }
    function uid() { return request.auth.uid; }
    function profile() { return get(/databases/{db}/documents/users/{uid()}).data; }
    function isActive() { return signedIn() && profile().status == 'active'; }
    function isAdmin() { return isActive() && profile().role == 'admin'; }
    function onlyChanges(keys) {
      return request.resource.data.diff(resource.data).affectedKeys().hasOnly(keys);
    }
    function listingOf(id) { return get(/databases/{db}/documents/listings/{id}).data; }

    // ... collection rules ...

    match /{document=**} { allow read, write: if false; } // deny-all fallback (keep last)
  }
}
```

BEST PRACTICE · Each `get()` in a rule costs one document read and counts toward the 10-call limit per request. Copy the fields you check most often (e.g., `ownerId` on bookings) onto the document being written, so the rules can check them without a `get()`.

3.2 Bookings: no client may confirm

```
match /bookings/{id} {
  allow read: if isAdmin() || (isActive() && uid() in [resource.data.renterId, resource.data.ownerId]);

  allow create: if isActive()
    && request.resource.data.renterId = uid()
    && request.resource.data.status = 'Pending'
    && listingOf(request.resource.data.listingId).verificationStatus = 'verified'
    && listingOf(request.resource.data.listingId).purpose = 'rent'
    && listingOf(request.resource.data.listingId).ownerId = request.resource.data.ownerId
    && request.resource.data.ownerId ≠ uid()
    && request.resource.data.startDate < request.resource.data.endDate;

  // Renter logs a deposit reference on a Confirmed booking - nothing else changes
  allow update: if isActive() && resource.data.renterId = uid()
    && resource.data.status = 'Confirmed' && request.resource.data.status = 'Confirmed'
    && onlyChanges(['depositReference', 'updatedAt']);
  // ... separate allow-update clauses for Completed and Disputed transitions ...
  // NOTE: no clause anywhere allows request.resource.data.status = 'Confirmed' from 'Pending'.

  allow delete: if isActive() && resource.data.status = 'Pending'
    && uid() in [resource.data.renterId, resource.data.ownerId];
}
```

3.3 Users: public versus private fields

A readable document exposes **every** field. If `users/{uid}` holds `email`, it must not be publicly readable. Recommended:

```
match /users/{userId} {
  allow read: if isAdmin() || (signedIn() && uid() = userId);
  allow create: if signedIn() && uid() = userId
    && request.resource.data.role = 'user' && request.resource.data.status = 'active';
  allow update: if (signedIn() && uid() = userId && onlyChanges(['name', 'photoUrl', 'updatedAt']))
    || (isAdmin() && onlyChanges(['status', 'updatedAt']));
}
match /publicProfiles/{userId} {           // name, photo, joinedAt only - written alongside users
  allow read: if true;
  allow write: if signedIn() && uid() = userId && onlyChanges(['name', 'photoUrl', 'updatedAt']);
}
```

WATCH OUT · Check what the deployed rules do for `users` today. If they allow public or any-signed-in read **and** the document contains an email, record it as a new finding. Then either remove the email from the document or apply the split above. Public profile pages and listing cards should read only `publicProfiles`.

3.4 Private listing data

```
match /listingPrivate/{id} {
  allow read: if isAdmin() || (isActive() && listingOf(id).ownerId == uid());
  // getAfter() lets the listing and its private doc be created in one batch
  allow create, update: if isActive()
    && getAfter(/databases/${db}/documents/listings/${id}).data.ownerId == uid();
  allow delete: if false;
}
```

3.5 Notifications feed boundary

```
match /notifications/{id} {
  allow read, update: if (signedIn() && resource.data.recipientId == uid() && (request.method == 'get'
  || request.method == 'list' || onlyChanges(['read'])))
    || (isAdmin() && resource.data.recipientId == '__admins__'
    && resource.data.type in ['dispute_opened', 'listing_submitted']);
  allow create: if isActive() && request.resource.data.read == false
    && ( (request.resource.data.recipientId == '__admins__' && request.resource.data.type in
  ['dispute_opened', 'listing_submitted'])
    || (request.resource.data.recipientId != '__admins__' && request.resource.data.type in
  ['booking_requested', 'booking_declined', 'listing_decision', 'dispute_resolved']) );
  allow delete: if signedIn() && resource.data.recipientId == uid();
}
```

These are **patterns, not drop-in code**. Merge them with the rules already deployed in commit [a1fe48e](#). Keep everything that already passes, and run the emulator suite after each change.

3.6 Testing rules properly

- Use [@firebase/rules-unit-testing](#) against the emulator. For every collection, write one **allowed** and one **denied** test per actor (guest, renter, owner, other user, suspended, admin).
- Test **writes that change one forbidden field** (for example, a renter adding `status: 'Confirmed'` alongside a valid deposit reference). These are the most common holes.
- Add the rules tests to CI so a careless change cannot be merged.

4. Trusted API hardening

Control	How	Why
Authentication	<code>verifyIdToken</code> on every non-health call; <code>checkRevoked: true</code> for document links	Stolen or old sessions
Authorization	Re-read <code>users/{uid}</code> for role/status; compare <code>ownerId</code> from Firestore, never from the request body	Clients can lie
Input validation	Allow-list values (<code>kind ∈ {ownership, govId}</code>); validate IDs as <code>^[A-Za-z0-9_-]{1,64}\$</code>	Injection, path tricks
CORS	Exact-match allow-list from <code>ALLOWED_ORIGINS</code>	Stops other websites calling the API from users' browsers
Rate limiting	Per-UID counter document (e.g., max 20 upload signatures/hour) checked in a transaction	Abuse of Cloudinary credits

Control	How	Why
Errors	Generic message for 500s; specific codes for 4xx	No stack traces or internals leaked
Logging	Log <code>uid</code> , endpoint, outcome, latency. Never tokens, URLs, addresses, document IDs	Logs are not a safe place for PII
Secrets	Vercel env vars marked Sensitive; separate Preview vs Production if possible	Limits the blast radius
Dependencies	Pin versions; <code>npm audit</code> in CI	Supply chain
Headers	<code>Cache-Control: no-store</code> , <code>X-Content-Type-Options: nosniff</code>	Document links must not be cached

5. Private verification documents (F-16)

5.1 Target state

- Documents are uploaded with a **server signature** to the `trustrhome_private_docs` preset as `authenticated` assets.
- Firestore stores only `{ publicId, format, resourceType }` in `listingPrivate/{id}.documents`, never a URL.
- Owner and admins view them through `GET /api/listings/{id}/documents/{kind}`, which returns a link valid for **5 minutes**.
- Public `listings` documents contain **no** document fields of any kind.

5.2 Migration of existing documents

The existing `bank_scraper/migrate_verification_documents.py` already moves URLs into `listingPrivate`. Extend it as follows. It must stay **dry-run by default**, idempotent, and print a report.

- Back up Firestore (03 §8).
- For each legacy document URL, parse the `public_id`, `resource_type` and format.
- Change the asset to authenticated: Cloudinary Admin API `rename(public_id, public_id, type='upload', to_type='authenticated', invalidate=True)`. The `invalidate` flag purges CDN copies of the old public URL.
- Write `{publicId, format, resourceType}` into `listingPrivate/{id}.documents.{kind}`.
- Remove every legacy URL field from `listings/{id}` (and from `listingPrivate` once the new fields verify).
- Report: migrated, already done, missing in Cloudinary (manual follow-up), errors.
- Verify: three old URLs no longer load (TC-DOC-008), and the public listing JSON has no document fields (TC-DOC-009).

WATCH OUT · CDN invalidation can take a few minutes. Test old URLs after about 10 minutes, and in a private browser window, so local cache does not fool you.

5.3 Admin review checklist (store the result with the decision)

- The name on the government ID matches the name on the ownership document. Minor variations are acceptable, such as middle initial or maiden name; note them.

2. The property address on the document matches the listing's city/area.
3. The document looks complete: no cut-off edges, no obvious edits, dates are plausible.
4. The ID is not expired.
5. The photos look consistent with the property type and size. Do a reverse-image check if something feels off.
6. Decision + note + reviewer UID + timestamp are saved. A rejection note tells the owner what to fix, without accusing them.

6. Credentials (F-01)

See *03 Infrastructure §6* for the procedure. Summary of the rules:

- One service account per consumer, with the **Cloud Datastore User** role only.
- Keys live only in Vercel and GitHub Secrets. Rotate them immediately after any exposure. Delete old keys, and prove the deletion with a failing login.
- Cloudinary API secret: only in Vercel. If it appears anywhere else, regenerate it in Cloudinary and update Vercel.
- Personal accounts: everyone on the Firebase, Cloudinary, Vercel and GitHub teams enables **2-step verification**.

7. Web app hardening (Firebase Hosting)

Add security headers in `firebase.json`:

```
"hosting": { "headers": [{ "source": "**", "headers": [
  { "key": "X-Frame-Options", "value": "DENY" },
  { "key": "X-Content-Type-Options", "value": "nosniff" },
  { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
  { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=(self)" },
  { "key": "Content-Security-Policy-Report-Only",
    "value": "default-src 'self'; img-src 'self' data: https://res.cloudinary.com
https://*.tile.openstreetmap.org; connect-src 'self' https://*.googleapis.com https://*.firebaseio.com
https://api.cloudinary.com https://*.vercel.app https://nominatim.openstreetmap.org; frame-src
https://*.firebaseapp.com https://accounts.google.com; script-src 'self' https://apis.google.com;
style-src 'self' 'unsafe-inline'" }
] ] }
```

Start the CSP in **Report-Only** mode, watch the browser console for violations during a full smoke test, adjust it, and only then switch to an enforcing `Content-Security-Policy`. Google sign-in and map tiles are the usual things that break.

Other front-end rules: never use `dangerouslySetInnerHTML` with user text; render descriptions and dispute reasons as plain text; give external links `rel="noopener noreferrer"`.

8. Data Privacy Act of 2012 (RA 10173) alignment

NOTE · This section is a practical checklist for a student project. It is not legal advice. For a real launch, have the privacy notice and processes reviewed by someone qualified in Philippine data-privacy law and current National Privacy Commission (NPC) issuances.

Government-issued identification is **sensitive personal information** under the Act. Processing it needs a lawful basis (typically the owner's **consent**, given for the specific purpose of listing verification) and stronger safeguards.

DPA principle / duty	What TrustHome does	Status
Transparency	Privacy Policy and Data & Compliance pages; consent text on the document-upload step stating purpose, who sees it, and retention	Pages exist; add an explicit consent checkbox at upload
Legitimate purpose	Documents used only for the verification decision	Policy
Proportionality	Only two documents; no ID numbers extracted or stored; exact address private	Done / v1.4
Security measures	Rules, private assets, short-lived links, least-privilege keys, 2-step verification	v1.4
Retention	Proposed: rejected listings: delete documents 30 days after decision. Verified listings: keep while active + 90 days after removal. Backups: delete after the defense	Team decision needed
Data-subject rights	Account page: download my data / delete my account request (manual admin process is acceptable for MVP)	Document the process
Breach handling	Who to call, what to contain, record keeping. The NPC expects notification of qualifying breaches within 72 hours of knowledge	Write a one-page incident plan
Accountability	Name a Data Protection Officer (team role) on the Data & Compliance page	Open (listed in progress report)

9. Security release gate

The release cannot be called "defense-ready" until all of these are true:

- ☐ F-05 closed: API deployed; TC-API-005/006 pass live; a direct client `Confirmed` write is denied.
- ☐ F-16 closed: new uploads are authenticated; migration applied; TC-DOC-001...009 pass.
- ☐ F-01 closed: old keys deleted; TC-OPS-001/002 recorded.
- ☐ Rules suite passes in the emulator and on the live project (TC-SEC-*).
- ☐ `users` read exposure checked (T10).
- ☐ No secrets in the frontend bundle: run `grep -R "api_secret\|PRIVATE KEY" dist/`, which must return nothing.
- ☐ Consent text present on the document upload step.